

main

June 4, 2026

```
[1]: import pandas as pd
```

In this code, I assemble the necessary data, clean it, conduct an exploratory data analysis, and then ...

1 Data Collection

1.1 Grid intensity

Grid intensity is important for manufacturing, but it's still relevant for supply chains. Source: [Ember Energy](#)

```
[2]: # gCO2/kWh - grid intensity is about CO2 released per unit of energy. Mostly
      ↪ about manufacturing, but still relevant
      grid_intensity = {"china": 525, "mexico": 412, "s_korea": 390}
```

1.2 Emission Factor

Ton-km emission factor is CO2 released per ton of commodity per kilometer transported. Relevant for transportation, depends on country's mix of transportation methods used. Because washing machines are transported mainly by trade vessels and trucks, these two are the main methods used in the calculation of the emission factor.

Lane-specific emission factors combine the IMO Fourth GHG Study global average with adjustments for typical vessel deployment on each lane (sourced from UNCTAD 2024 Chapter II) and feeder-megaship transshipment patterns documented in Notteboom & Rodrigue (2009).

Sources: + [US EPA SmartWay Carrier Emission Factors](#) - emission factor of Mexico-US trade routes + [New shipping routes highlight growing Asia-to-Mexico trade](#) - emission factor of Asia-Pacific trade routes + [Review of Maritime Transport 2024: Navigating Maritime Chokepoints](#) - GHG global average with country-specific adjustments for each trade route. + [Notteboom and Rodrigue](#) - documents shipment patterns of feeder megaships

```
[3]: # gCO2/ton-km - emission factor is CO2 released per ton of commodity per
      ↪ kilometer transported. Different countries use different methods

      # ton_km = {
      #     "china": 4,
      #     "south_korea": 4.5,
      #     "vietnam": 7,
```

```
#     "india": 6,
#     "mexico": 80
# }
```

```
ton_km = {
    "5700": 4,
    "5800": 4.5,
    "5520": 7,
    "5330": 6,
    "2010": 80
}
```

[]:

1.3 Distance

This code calculates distances between ports for countries beyond the ocean(China, India, South Korea, Vietnam), as well as land distance over the US border for Mexico.

1.3.1 ISTHS6M

```
[4]: # first, analyze the files from census.gov site. They're encoded using
      ↪fixed-width ASCII, which makes it hard for conventional Pandas functions to
      ↪handle it
      # for this reason, I had to enlist Claude's help to solve it.

      # Correct colspecs for ISTHS6M (State HS6 Imports), record length 258
      colspecs = [
          (0, 6),      # commodity (6-digit HS code)
          (6, 10),     # cty_code (4-digit country code)
          (10, 12),    # state (2-letter postal)
          (12, 16),    # year
          (16, 18),    # month
          (18, 33),    # gen_val_mo (general imports total value)
          (33, 48),    # con_val_mo (imports for consumption value)
          (48, 63),    # air_val_mo
          (63, 78),    # air_swt_mo (air shipping weight, kg)
          (78, 93),    # ves_val_mo
          (93, 108),   # ves_swt_mo (vessel shipping weight, kg)
          (108, 123),  # cnt_val_mo (containerized vessel value)
          (123, 138),  # cnt_swt_mo (containerized vessel weight)
          (138, 153),  # gen_val_yr (year-to-date)
          (153, 168),  # con_val_yr
          (168, 183),  # air_val_yr
          (183, 198),  # air_swt_yr
          (198, 213),  # ves_val_yr
          (213, 228),  # ves_swt_yr
```

```

    (228, 243), # cnt_val_yr
    (243, 258), # cnt_swt_yr
]
names = ['commodity', 'cty_code', 'state', 'year', 'month',
         'gen_val_mo', 'con_val_mo', 'air_val_mo', 'air_swt_mo',
         'ves_val_mo', 'ves_swt_mo', 'cnt_val_mo', 'cnt_swt_mo',
         'gen_val_yr', 'con_val_yr', 'air_val_yr', 'air_swt_yr',
         'ves_val_yr', 'ves_swt_yr', 'cnt_val_yr', 'cnt_swt_yr']

str_cols = ['commodity', 'cty_code', 'state', 'year', 'month']

df_land_imports = pd.read_fwf(
    'data/original/isthsm2501.txt',
    colspecs=colspecs,
    names=names,
    dtype={c: str for c in str_cols},
)

```

```
[5]: df_land_imports.head()
```

```

[5]: commodity cty_code state year month gen_val_mo con_val_mo air_val_mo \
0    010121    1220    FL  2025    01      57550      57550        0
1    010121    1220    KY  2025    01      28770      28770        0
2    010121    1220    MI  2025    01      11500      11500        0
3    010121    1220    MO  2025    01      10000      10000        0
4    010121    1220    MT  2025    01      25000      25000        0

    air_swt_mo ves_val_mo ... cnt_val_mo cnt_swt_mo gen_val_yr \
0           0         0 ...          0          0      57550
1           0         0 ...          0          0      28770
2           0         0 ...          0          0      11500
3           0         0 ...          0          0      10000
4           0         0 ...          0          0      25000

    con_val_yr air_val_yr air_swt_yr ves_val_yr ves_swt_yr cnt_val_yr \
0      57550         0         0         0         0         0
1      28770         0         0         0         0         0
2      11500         0         0         0         0         0
3      10000         0         0         0         0         0
4       2500         0         0         0         0         0

    cnt_swt_yr
0           0
1           0
2           0
3           0
4           0

```

[5 rows x 21 columns]

```
[6]: df_land_imports.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 449110 entries, 0 to 449109
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   commodity             449110 non-null  object
1   cty_code              449110 non-null  object
2   state                 449110 non-null  object
3   year                  449110 non-null  object
4   month                 449110 non-null  object
5   gen_val_mo            449110 non-null  int64
6   con_val_mo            449110 non-null  int64
7   air_val_mo            449110 non-null  int64
8   air_swt_mo           449110 non-null  int64
9   ves_val_mo            449110 non-null  int64
10  ves_swt_mo            449110 non-null  int64
11  cnt_val_mo            449110 non-null  int64
12  cnt_swt_mo            449110 non-null  int64
13  gen_val_yr            449110 non-null  int64
14  con_val_yr            449110 non-null  int64
15  air_val_yr            449110 non-null  int64
16  air_swt_yr            449110 non-null  int64
17  ves_val_yr            449110 non-null  int64
18  ves_swt_yr            449110 non-null  int64
19  cnt_val_yr            449110 non-null  int64
20  cnt_swt_yr            449110 non-null  int64
dtypes: int64(16), object(5)
memory usage: 72.0+ MB
```

This is the code that could be used to auto-import data. However, the problem with it is that there is poor connection between the API and the website.œ

```
[7]: # import requests
# from pathlib import Path

# OUT = Path("data/port_hs6")
# OUT.mkdir(parents=True, exist_ok=True)

# def url(year, month):
#     yy = str(year)[2:]
#     return (f"https://www.census.gov/trade/downloads/{year}"
#            f"/Port/im_hs6_m/PORTHS6MM{yy}{month:02d}.ZIP")
```

```

# # Your tariff treatment window
# for year in (2023, 2024, 2025):
#     for month in range(1, 13):
#         u = url(year, month)
#         out_path = OUT / f"PORTHS6MM{str(year)[2:]}{month:02d}.ZIP"
#         if out_path.exists():
#             continue
#         r = requests.get(u, timeout=30)
#         if r.status_code == 200:
#             out_path.write_bytes(r.content)
#             print(f"saved {out_path}")
#         else:
#             print(f"missing or not yet released: {u}")

```

1.3.2 PORTHS6MM

```

[8]: # PORTHS6MM (Port HS6 Imports) record layout - record length 230
# Source: census.gov/foreign-trade/reference/products/layouts/dporths6i.html
colspecs = [
    (0, 6),      # commodity (6-digit HS code)
    (6, 10),     # cty_code (Schedule C, 4-digit)
    (10, 12),    # dist_unlade (Schedule D district, 2-digit)
    (12, 14),    # port_unlade (Schedule D port within district, 2-digit)
    (14, 18),    # year
    (18, 20),    # month
    (20, 35),    # gen_val_mo (general imports value, this month)
    (35, 50),    # air_val_mo
    (50, 65),    # air_swt_mo (air shipping weight, kg)
    (65, 80),    # ves_val_mo
    (80, 95),    # ves_swt_mo (vessel shipping weight, kg)
    (95, 110),   # cnt_val_mo (containerized vessel value)
    (110, 125),  # cnt_swt_mo (containerized vessel weight)
    (125, 140),  # gen_val_yr (year-to-date totals begin)
    (140, 155),  # air_val_yr
    (155, 170),  # air_swt_yr
    (170, 185),  # ves_val_yr
    (185, 200),  # ves_swt_yr
    (200, 215),  # cnt_val_yr
    (215, 230),  # cnt_swt_yr
]
names = ['commodity', 'cty_code', 'dist_unlade', 'port_unlade', 'year', 'month',
         'gen_val_mo', 'air_val_mo', 'air_swt_mo', 'ves_val_mo', 'ves_swt_mo',
         'cnt_val_mo', 'cnt_swt_mo',
         'gen_val_yr', 'air_val_yr', 'air_swt_yr', 'ves_val_yr', 'ves_swt_yr',
         'cnt_val_yr', 'cnt_swt_yr']
str_cols = ['commodity', 'cty_code', 'dist_unlade', 'port_unlade', 'year', 'month']

```

```

df_ocean_routes = pd.read_fwf(
    'data/original/PORTHS6MM2501.TXT',
    colspecs=colspecs,
    names=names,
    dtype={c: str for c in str_cols},
)

# Build the full 4-digit Schedule D port code (district + port)
df_ocean_routes['port_full'] = df_ocean_routes['dist_unlade'] +
    df_ocean_routes['port_unlade']

```

https://www.census.gov/trade/downloads/2025/state_imp/hs6_m/ISTHSM2512.ZIP

```

[9]: import searoute as sr

# coordinates as [lon, lat]
# Shanghai → LA
route = sr.searoute([121.47, 31.23], [-118.27, 33.74])
# in km, following real sea lanes
distance_km = route.properties['length']

```

1.4 Weight of imports

Similar to UN Comtrade, but focuses on the USA and is more laconic.

USA Trade Census + Data Source Selection: State Import Data(Harmonized System) + Filters: + Measures: Vessel SWT and Air SWT(kg) - The gross weight in kilograms of shipments made by seafaring vessel/airplane at customs + No data on land transportation there + State: All States + Commodity: 845011, 845012, 845019 and 845020(washing machines) + Country: India, South Korea, Mexico + Time: Jan 2025 - Mar 2026(monthly)

```

[10]: washing_machine_trade_filepath = "data/original/State Imports by HS
    ↪Commodities_v4.csv"
washing_machine_df = pd.read_csv(washing_machine_trade_filepath,
    ↪index_col=False, header=2)

```

```

[11]: washing_machine_df.head()

```

```

[11]:

```

	Commodity	Country	Time \
0	845011 Washing Mach Automatic W Dry Line Cap N...	China	January 2025
1	845011 Washing Mach Automatic W Dry Line Cap N...	China	February 2025
2	845011 Washing Mach Automatic W Dry Line Cap N...	China	March 2025
3	845011 Washing Mach Automatic W Dry Line Cap N...	China	April 2025
4	845011 Washing Mach Automatic W Dry Line Cap N...	China	May 2025

	Air SWT (kg)	Vessel SWT (kg)	Unnamed: 5
0	174	1,777,528	NaN

1	NaN	2,459,827	NaN
2	NaN	2,268,490	NaN
3	NaN	2,512,291	NaN
4	NaN	2,541,012	NaN

2 Data Cleaning

Cleaning is the longest and the most important step of any data cycle. After all, without good data there can be no good results. Because this projects uses data from a wide variety of sources, this means dealing with many different formats, which may complicate the cleaning process even further.

2.1 Land-based Distance/Weight Data(Mexico) - US Census

No dataset has everything we need. However, if we gather from enough data sources, we might get a complete picture. But without a good cleaning, inconsistencies will ruin things.

```
[12]: df_land_imports_mex = df_land_imports[df_land_imports["cty_code"] == "2010"]
```

```
[13]: df_land_imports_mex_v1 = df_land_imports_mex.T.drop_duplicates().T
df_land_imports_mex_v1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 19036 entries, 67 to 448890
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   commodity       19036 non-null  object
1   cty_code        19036 non-null  object
2   state          19036 non-null  object
3   year           19036 non-null  object
4   month          19036 non-null  object
5   gen_val_mo     19036 non-null  object
6   con_val_mo     19036 non-null  object
7   air_val_mo     19036 non-null  object
8   air_swt_mo     19036 non-null  object
9   ves_val_mo     19036 non-null  object
10  ves_swt_mo     19036 non-null  object
11  cnt_val_mo     19036 non-null  object
12  cnt_swt_mo     19036 non-null  object
dtypes: object(13)
memory usage: 2.5+ MB
```

```
[14]: numerical_cols = ["gen_val_mo", "con_val_mo", "air_val_mo", "air_swt_mo",
    ↪ "ves_val_mo", "ves_swt_mo", "cnt_val_mo", "cnt_swt_mo"]

for col in numerical_cols:
    df_land_imports_mex_v1[col] = pd.to_numeric(df_land_imports_mex_v1[col])
```

```
df_land_imports_mex_v1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 19036 entries, 67 to 448890
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   commodity        19036 non-null  object
1   cty_code          19036 non-null  object
2   state            19036 non-null  object
3   year             19036 non-null  object
4   month            19036 non-null  object
5   gen_val_mo        19036 non-null  int64
6   con_val_mo        19036 non-null  int64
7   air_val_mo        19036 non-null  int64
8   air_swt_mo        19036 non-null  int64
9   ves_val_mo        19036 non-null  int64
10  ves_swt_mo        19036 non-null  int64
11  cnt_val_mo        19036 non-null  int64
12  cnt_swt_mo        19036 non-null  int64
dtypes: int64(8), object(5)
memory usage: 2.5+ MB
```

```
[15]: df_land_imports_mex_wash =
↳ df_land_imports_mex_v1[df_land_imports_mex_v1["commodity"].isin(["845011",
↳ "845020"])]
df_land_imports_mex_wash.describe()
```

```
[15]:
```

	gen_val_mo	con_val_mo	air_val_mo	air_swt_mo	ves_val_mo	\
count	2.300000e+01	2.300000e+01	23.0	23.0	23.000000	
mean	1.290286e+06	1.289363e+06	0.0	0.0	25014.434783	
std	2.927376e+06	2.927754e+06	0.0	0.0	47158.716741	
min	1.684900e+04	1.684900e+04	0.0	0.0	0.000000	
25%	4.376000e+04	4.376000e+04	0.0	0.0	0.000000	
50%	1.882560e+05	1.882560e+05	0.0	0.0	0.000000	
75%	1.176076e+06	1.176076e+06	0.0	0.0	25537.500000	
max	1.376476e+07	1.376476e+07	0.0	0.0	146506.000000	

	ves_swt_mo	cnt_val_mo	cnt_swt_mo
count	23.000000	23.000000	23.000000
mean	5846.434783	25014.434783	5846.434783
std	11152.366063	47158.716741	11152.366063
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000
75%	6185.000000	25537.500000	6185.000000
max	33517.000000	146506.000000	33517.000000

So, upon filtering and cleaning data, we recognize that the vast majority of imports of Mexican washing machines into the USA is done through the land, rather than sea or plane. This confirmed my initial hypothesis on Mexico's transformation breakdown and now justifies the plan to calculate solely the inland leg's carbon footprint for imports from Mexico.

Now, we need to understand the distances from each state to state.

```
[16]: # import requests
# from pathlib import Path

# OUT = Path("data/port_hs6")
# OUT.mkdir(parents=True, exist_ok=True)

# def url(year, month):
#     yy = str(year)[2:]
#     return (f"https://www.census.gov/trade/downloads/{year}"
#            f"/Port/im_hs6_m/PORTHS6MM{yy}{month:02d}.ZIP")

# # Your tariff treatment window
# for year in (2023, 2024, 2025):
#     for month in range(1, 13):
#         u = url(year, month)
#         out_path = OUT / f"PORTHS6MM{str(year)[2:]}{month:02d}.ZIP"
#         if out_path.exists():
#             continue
#         r = requests.get(u, timeout=30)
#         if r.status_code == 200:
#             out_path.write_bytes(r.content)
#             print(f"saved {out_path}")
#         else:
#             print(f"missing or not yet released: {u}")
```

A dictionary of coordinates is necessary. I calculate starting from Monterrey, Nuevo León, since that is where most of domestic appliance manufacturing is located.

```
[17]: mexico_start = [-100.310, 25.671]    # Monterrey, Nuevo León

state_centroids = {
    "CA": [-119.418, 36.778],
    "CO": [-105.548, 38.998],
    "FL": [-81.516, 27.664],
    "GA": [-82.907, 32.157],
    "IL": [-88.986, 40.349],
    "IN": [-86.258, 39.849],
    "KY": [-84.670, 37.668],
    "MD": [-76.802, 39.063],
    "MI": [-85.602, 44.314],
    "NJ": [-74.521, 40.298],
```

```

    "OH": [ -82.764, 40.388],
    "PA": [ -77.210, 40.590],
    "PR": [ -66.591, 18.220],
    "TX": [ -97.564, 31.054],
    "WA": [-121.490, 47.400],
}

```

```

[18]: from geopy.distance import great_circle

def compute_distance_land(row):
    origin = mexico_start[::-1]
    dest = state_centroids[row["state"]][::-1]
    return great_circle(origin, dest).km * 1.3

df_land_imports_mex_wash_v1 = df_land_imports_mex_wash.copy()

df_land_imports_mex_wash_v1["distance"] = df_land_imports_mex_wash_v1.
    ↪apply(compute_distance_land, axis=1)

```

```

[19]: df_land_imports_mex_wash.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 23 entries, 296915 to 297007
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   commodity       23 non-null    object
 1   cty_code        23 non-null    object
 2   state           23 non-null    object
 3   year            23 non-null    object
 4   month           23 non-null    object
 5   gen_val_mo      23 non-null    int64
 6   con_val_mo      23 non-null    int64
 7   air_val_mo      23 non-null    int64
 8   air_swt_mo      23 non-null    int64
 9   ves_val_mo      23 non-null    int64
10   ves_swt_mo      23 non-null    int64
11   cnt_val_mo      23 non-null    int64
12   cnt_swt_mo      23 non-null    int64
dtypes: int64(8), object(5)
memory usage: 2.5+ KB

```

```

[20]: df_land_imports_mex_wash_v1.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 23 entries, 296915 to 297007
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype

```

```

---  -----  -----  -----
0  commodity    23 non-null    object
1  cty_code     23 non-null    object
2  state        23 non-null    object
3  year         23 non-null    object
4  month        23 non-null    object
5  gen_val_mo   23 non-null    int64
6  con_val_mo   23 non-null    int64
7  air_val_mo   23 non-null    int64
8  air_swt_mo   23 non-null    int64
9  ves_val_mo   23 non-null    int64
10 ves_swt_mo   23 non-null    int64
11 cnt_val_mo   23 non-null    int64
12 cnt_swt_mo   23 non-null    int64
13 distance     23 non-null    float64
dtypes: float64(1), int64(8), object(5)
memory usage: 2.7+ KB

```

```
[21]: df_land_imports_mex_wash_v1.to_csv("data/result/ISTHS6MM_mex.csv")
```

2.2 Ocean-based Distance/Weight data

```
[22]: asian_countries_codes = ["5700", "5800", "5520", "5330"]
df_asian_routes = df_ocean_routes[df_ocean_routes["cty_code"].
    ↪isin(asian_countries_codes)]
```

```
[23]: df_asian_routes_v1 = df_asian_routes.T.drop_duplicates().T
```

```
[24]: df_asian_routes_v1.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 102949 entries, 108 to 418323
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   commodity       102949 non-null object
1   cty_code        102949 non-null object
2   dist_unlade     102949 non-null object
3   port_unlade     102949 non-null object
4   year            102949 non-null object
5   month           102949 non-null object
6   gen_val_mo      102949 non-null object
7   air_val_mo      102949 non-null object
8   air_swt_mo      102949 non-null object
9   ves_val_mo      102949 non-null object
10  ves_swt_mo      102949 non-null object
11  cnt_val_mo      102949 non-null object
12  cnt_swt_mo      102949 non-null object

```

```

13 port_full      102949 non-null object
dtypes: object(14)
memory usage: 15.8+ MB

```

```
[25]: df_asian_routes_v1.head()
```

```

[25]: commodity cty_code dist_unlade port_unlade year month gen_val_mo \
108 010129 5800 39 01 2025 01 10000
170 010611 5520 10 12 2025 01 7920000
171 010611 5520 54 01 2025 01 7500000
287 010619 5330 17 04 2025 01 3500
288 010619 5700 04 17 2025 01 370924

air_val_mo air_swt_mo ves_val_mo ves_swt_mo cnt_val_mo cnt_swt_mo \
108 10000 700 0 0 0 0
170 7920000 7360 0 0 0 0
171 7500000 4284 0 0 0 0
287 3500 57 0 0 0 0
288 370924 369 0 0 0 0

port_full
108 3901
170 1012
171 5401
287 1704
288 0417

```

```
[26]: numerical_cols_asian = ["gen_val_mo", "air_val_mo", "air_swt_mo", "ves_val_mo",
    ↪ "ves_swt_mo", "cnt_val_mo", "cnt_swt_mo"]
```

```

for col in numerical_cols_asian:
    df_asian_routes_v1[col] = pd.to_numeric(df_asian_routes_v1[col])
df_asian_routes_v1.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 102949 entries, 108 to 418323
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   commodity       102949 non-null object
1   cty_code        102949 non-null object
2   dist_unlade     102949 non-null object
3   port_unlade     102949 non-null object
4   year            102949 non-null object
5   month           102949 non-null object
6   gen_val_mo      102949 non-null int64
7   air_val_mo      102949 non-null int64
8   air_swt_mo      102949 non-null int64

```

```

9   ves_val_mo    102949 non-null   int64
10  ves_swt_mo    102949 non-null   int64
11  cnt_val_mo    102949 non-null   int64
12  cnt_swt_mo    102949 non-null   int64
13  port_full     102949 non-null   object
dtypes: int64(7), object(7)
memory usage: 15.8+ MB

```

```

[27]: df_asian_routes_wash = df_asian_routes_v1[df_asian_routes_v1["commodity"].
      ↪isin(["845011", "845020"])]
df_asian_routes_wash.head()

```

```

[27]: commodity cty_code dist_unlade port_unlade year month gen_val_mo \
273372  845011    5520         10         03  2025    01    1476672
273373  845011    5520         13         03  2025    01         3888
273374  845011    5520         14         01  2025    01         8976
273375  845011    5520         17         03  2025    01    153763
273376  845011    5520         27         04  2025    01    19801

      air_val_mo air_swt_mo ves_val_mo ves_swt_mo cnt_val_mo \
273372         0         0    1476672    215096    1476672
273373         0         0         3888         740         3888
273374         0         0         8976        2224         8976
273375         0         0    153763     21411    153763
273376         0         0     19801      4583     19801

      cnt_swt_mo port_full
273372    215096     1003
273373         740     1303
273374     2224     1401
273375     21411     1703
273376     4583     2704

```

I will define dictionaries for country codes of origin country(“`cty_code`”) and codes representing US ports of arrival(“`port_full`”)

I assume that each country has one main port, from which it exports its goods.

```

[28]: asian_origins = {
      "5700": [121.470, 31.230],
      "5800": [129.040, 35.100],
      "5520": [106.780, 10.780],
      "5330": [ 69.710, 22.740],
      }

# US ports of unloading (Schedule D 4-digit code -> [lon, lat])
us_ports = {
      "1003": [ -74.165, 40.685], # Newark, NJ (Port Newark/Elizabeth)

```

```

"1303": [ -76.535, 39.265], # Baltimore, MD (Seagirt Marine Terminal)
"1401": [ -76.330, 36.880], # Norfolk/Newport News, VA
"1601": [ -79.911, 32.852], # Charleston, SC
"1703": [ -81.140, 32.115], # Savannah, GA (Garden City Terminal)
"1801": [ -82.456, 27.910], # Tampa, FL
"1803": [ -81.534, 30.395], # Jacksonville, FL (Blount Island/Dames Point)
"1901": [ -88.040, 30.690], # Mobile, AL
"2002": [ -90.060, 29.940], # New Orleans, LA
"2704": [-118.265, 33.733], # Los Angeles, CA
"2709": [-118.216, 33.755], # Long Beach, CA
"2809": [-122.395, 37.795], # San Francisco, CA
"2811": [-122.290, 37.795], # Oakland, CA
"2904": [-122.755, 45.585], # Portland, OR
"3001": [-122.330, 47.610], # Seattle, WA
"3002": [-122.430, 47.275], # Tacoma, WA
"3126": [-149.890, 61.235], # Anchorage, AK
"3201": [-157.870, 21.310], # Honolulu, HI
"3901": [ -87.620, 41.880], # Chicago, IL (Great Lakes / inland)
"4101": [ -81.700, 41.500], # Cleveland, OH (Great Lakes)
"4909": [ -66.100, 18.450], # San Juan, PR
"5201": [ -80.165, 25.778], # Miami, FL (PortMiami)
"5203": [ -80.116, 26.090], # Port Everglades, FL
"5206": [ -80.290, 25.796], # Miami International AIRPORT - exclude from
↪ sea analysis
"5301": [ -95.020, 29.733], # Houston, TX
}

```

```
[29]: asian_origins["5700"]
```

```
[29]: [121.47, 31.23]
```

```

[30]: import searoute as sr

# coordinates as [lon, lat]
# Shanghai → LA
route = sr.searoute([121.47, 31.23], [-118.27, 33.74])
# in km, following real sea lanes
distance_km = route.properties['length']

```

As there are only 76 rows in the final dataframe, it's possible to iterate over them by applying the distance-calculation function to each row.

```

[31]: def compute_distance_ocean(row):
    origin = asian_origins[row["cty_code"]]
    dest = us_ports[row["port_full"]]
    return sr.searoute(origin, dest).properties["length"]

```

```
df_asian_routes_wash["distance"] = df_asian_routes_wash.  
    ↪ apply(compute_distance_ocean, axis=1)
```

/tmp/ipykernel_17068/45734114.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_asian_routes_wash["distance"] =  
df_asian_routes_wash.apply(compute_distance_ocean, axis=1)
```

```
[32]: df_asian_routes_wash.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
Int64Index: 76 entries, 273372 to 273506  
Data columns (total 15 columns):  
#   Column          Non-Null Count  Dtype  
---  -  
0   commodity        76 non-null    object  
1   cty_code          76 non-null    object  
2   dist_unlade       76 non-null    object  
3   port_unlade       76 non-null    object  
4   year              76 non-null    object  
5   month             76 non-null    object  
6   gen_val_mo        76 non-null    int64  
7   air_val_mo        76 non-null    int64  
8   air_swt_mo        76 non-null    int64  
9   ves_val_mo        76 non-null    int64  
10  ves_swt_mo        76 non-null    int64  
11  cnt_val_mo        76 non-null    int64  
12  cnt_swt_mo        76 non-null    int64  
13  port_full         76 non-null    object  
14  distance          76 non-null    float64  
dtypes: float64(1), int64(7), object(7)  
memory usage: 9.5+ KB
```

```
[33]: df_asian_routes_wash.to_csv("data/result/PORTHS6MM_asian.csv")
```

2.3 Weight Data(USA Trade Online) - irrelevant?

Already well-organized, the weight data does not need much cleaning.

However, the previous datasets(US Census) contains everything necessary. Not just weight of imports, but also origin and destination info. Use this as a sanity check for weight of ocean-side imports.

```
[34]: washing_machine_df_v1 = washing_machine_df.drop("Unnamed: 5", axis=1)
```

```
washing_machine_df_v1["Air SWT (kg)"] = washing_machine_df_v1["Air SWT (kg)"].
↳str.replace(',', '')
washing_machine_df_v1["Vessel SWT (kg)"] = washing_machine_df_v1["Vessel SWT_
↳(kg)"].str.replace(',', '')

washing_machine_df_v1["Air SWT (kg)"] = pd.
↳to_numeric(washing_machine_df_v1["Air SWT (kg)"])
washing_machine_df_v1["Vessel SWT (kg)"] = pd.
↳to_numeric(washing_machine_df_v1["Vessel SWT (kg)"])

washing_machine_df_v1 = washing_machine_df_v1.fillna(0)
```

```
[35]: washing_machine_df_v1.head()
```

```
[35]:
```

		Commodity	Country	Time \
0	845011	Washing Mach Automatic W Dry Line Cap N...	China	January 2025
1	845011	Washing Mach Automatic W Dry Line Cap N...	China	February 2025
2	845011	Washing Mach Automatic W Dry Line Cap N...	China	March 2025
3	845011	Washing Mach Automatic W Dry Line Cap N...	China	April 2025
4	845011	Washing Mach Automatic W Dry Line Cap N...	China	May 2025

	Air SWT (kg)	Vessel SWT (kg)
0	174.0	1777528.0
1	0.0	2459827.0
2	0.0	2268490.0
3	0.0	2512291.0
4	0.0	2541012.0

```
[36]: washing_machine_df_v1[washing_machine_df_v1["Country"]=="Mexico"].head()
```

```
[36]:
```

		Commodity	Country	Time \
32	845011	Washing Mach Automatic W Dry Line Cap N...	Mexico	January 2025
33	845011	Washing Mach Automatic W Dry Line Cap N...	Mexico	February 2025
34	845011	Washing Mach Automatic W Dry Line Cap N...	Mexico	March 2025
35	845011	Washing Mach Automatic W Dry Line Cap N...	Mexico	April 2025
36	845011	Washing Mach Automatic W Dry Line Cap N...	Mexico	May 2025

	Air SWT (kg)	Vessel SWT (kg)
32	0.0	55246.0
33	0.0	74445.0
34	0.0	239852.0
35	0.0	175512.0
36	0.0	199598.0

```
[37]: washing_machine_df_v1["Time"].value_counts()
```



```
[37]: April 2025          9
      June 2025          9
      July 2025          9
      October 2025       9
      December 2025      9
      2026 through March 9
      January 2026       9
      January 2025       8
      February 2025      8
      March 2025         8
      May 2025           8
      August 2025        8
      September 2025     8
      November 2025      8
      February 2026      8
      March 2026         8
      Name: Time, dtype: int64
```

```
[38]: washing_machine_df_v1["Air SWT (kg)"].mean()
```

```
[38]: 171.28148148148148
```

3 CO2 Calculations(transportation)

Now that we have the three necessary components - ton-km CO2 factor, weight of imports and distance of imports, then we can calculate the carbon footprint of transportation. This is only one step, as we also need to calculate CO2 of manufacturing(grid intensity x energy spent on manufacturing) in order to understand the full extent of the carbon footprint.

CO2, in this case, is measured in grams. For now, we do this only for 5 countries(China, Vietnam, South Korea, India, Mexico) and only for the month of January 2025, using the datasets taken from census.gov website.

```
[ ]: def compute_co2(row):
      weight = df_land_imports_mex_wash_v1[row["cnt_swt_mo"]]
      dist = df_land_imports_mex_wash_v1[row["distance"]]
      co2_coeff = ton_km[df_land_imports_mex_wash_v1[row["cty_code"]]]
      return weight * dist * co2_coeff

df_land_imports_mex_wash_v1["co2"] = df_land_imports_mex_wash_v1.
    ↪apply(compute_co2, axis=1)
```

```
-----
KeyError                                Traceback (most recent call last)
File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
    ↪python3.11/site-packages/pandas/core/indexes/base.py:3802, in Index.
    ↪get_loc(self, key, method, tolerance)
    3801 try:
```

```
-> 3802     return self._engine.get_loc(casted_key)
    3803 except KeyError as err:
```

```
File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
↳python3.11/site-packages/pandas/_libs/index.pyx:138, in pandas._libs.index.
↳IndexEngine.get_loc()
```

```
File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
↳python3.11/site-packages/pandas/_libs/index.pyx:165, in pandas._libs.index.
↳IndexEngine.get_loc()
```

```
File pandas/_libs/hashtable_class_helper.pxi:5745, in pandas._libs.hashtable.
↳PyObjectHashTable.get_item()
```

```
File pandas/_libs/hashtable_class_helper.pxi:5753, in pandas._libs.hashtable.
↳PyObjectHashTable.get_item()
```

KeyError: 0

The above exception was the direct cause of the following exception:

KeyError Traceback (most recent call last)

Cell In[45], line 7

```
4     co2_coeff = ton_km[df_land_imports_mex_wash_v1[row["cty_code"]]]
5     return weight
----> 7 df_land_imports_mex_wash_v1["co2"] =_
↳df_land_imports_mex_wash_v1.apply(compute_co2, axis=1)
```

```
File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
↳python3.11/site-packages/pandas/core/frame.py:9568, in DataFrame.apply(self,
↳func, axis, raw, result_type, args, **kwargs)
```

```
9557 from pandas.core.apply import frame_apply
9559 op = frame_apply(
9560     self,
9561     func=func,
9562     (...) 9566     kwargs=kwargs,
9567 )
-> 9568 return op.apply().__finalize__(self, method="apply")
```

```
File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
↳python3.11/site-packages/pandas/core/apply.py:764, in FrameApply.apply(self)
```

```
761 elif self.raw:
762     return self.apply_raw()
--> 764 return self.apply_standard()
```

```
File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
↳python3.11/site-packages/pandas/core/apply.py:891, in FrameApply.
↳apply_standard(self)
```

```
890 def apply_standard(self):
```

```

--> 891     results, res_index = self.apply_series_generator()
      893     # wrap results
      894     return self.wrap_results(results, res_index)

File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
python3.11/site-packages/pandas/core/apply.py:907, in FrameApply.
    apply_series_generator(self)
      904 with option_context("mode.chained_assignment", None):
      905     for i, v in enumerate(series_gen):
      906         # ignore SettingWithCopy here in case the user mutates
--> 907         results[i] = self.f(v)
      908         if isinstance(results[i], ABCSeries):
      909             # If we have a view on v, we need to make a copy because
      910             # series_generator will swap out the underlying data
      911             results[i] = results[i].copy(deep=False)

```

```

Cell In[45], line 2, in compute_co2(row)
      1 def compute_co2(row):
----> 2     weight = df_land_imports_mex_wash_v1[row[      ]]
      3     dist = df_land_imports_mex_wash_v1[row["distance"]]
      4     co2_coeff = ton_km[df_land_imports_mex_wash_v1[row["cty_code"]]]

```

```

File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
python3.11/site-packages/pandas/core/frame.py:3807, in DataFrame.
    __getitem__(self, key)
      3805 if self.columns.nlevels > 1:
      3806     return self._getitem_multilevel(key)
-> 3807 indexer = self.columns.get_loc(key)
      3808 if is_integer(indexer):
      3809     indexer = [indexer]

```

```

File ~/Downloads/Medium_Secret/University/Programming/Python/src/venvol/lib/
python3.11/site-packages/pandas/core/indexes/base.py:3804, in Index.
    get_loc(self, key, method, tolerance)
      3802     return self._engine.get_loc(casted_key)
      3803 except KeyError as err:
-> 3804     raise KeyError(key) from err
      3805 except TypeError:
      3806     # If we have a listlike key, _check_indexing_error will raise
      3807     # InvalidIndexError. Otherwise we fall through and re-raise
      3808     # the TypeError.
      3809     self._check_indexing_error(key)

```

KeyError: 0

```
[40]: df_land_imports_mex_wash_v1.head()
```

```
[40]: commodity cty_code state year month gen_val_mo con_val_mo \
296915 845011 2010 CA 2025 01 2611237 2611237
296916 845011 2010 CO 2025 01 333416 333416
296917 845011 2010 FL 2025 01 1368227 1368227
296918 845011 2010 GA 2025 01 617518 617518
296919 845011 2010 IN 2025 01 983925 983925
```

```
air_val_mo air_swt_mo ves_val_mo ves_swt_mo cnt_val_mo \
296915 0 0 0 0 0
296916 0 0 0 0 0
296917 0 0 50281 11227 50281
296918 0 0 0 0 0
296919 0 0 0 0 0
```

```
cnt_swt_mo distance co2
296915 0 2847.115733 NaN
296916 0 2028.894380 NaN
296917 11227 2442.374793 NaN
296918 0 2389.453884 NaN
296919 0 2660.910286 NaN
```

```
[41]: # df_asian_routes_wash["co2"] =
↳ df_asian_routes_wash["vessel_swt_mo"]*df_asian_routes_wash["distance"]*ton_km[df_asian_rout
```